

Test Plan: Northstar Application Backend API

1 Introduction

1.1 Purpose and scope of document

This document outlines the testing strategy and validation process for the Northstar application's backend API system. The objective is to ensure that all API endpoints function as intended, authentication mechanisms work correctly, and database operations are reliable.

The scope of testing includes the following core functionalities:

- Authentication and Authorization
- Profile management
- Segments and Reviews
- Help Requests and Responses
- Frontend testing
- Load or performance testing

1.2 Product and environment

The product is a backend system developed using Node.js, TypeScript, and Express. Data management is handled via Prisma ORM with a PostgreSQL database.

The testing environment consists of:

- **Local Environment:** For development and manual verification.
- **Continuous Integration (CI) Environment:** Utilizing GitHub Actions for automated checks.
- **Testing Frameworks:** Vitest is used for unit testing.

1.3 Project constraints related to testing

- **Database Isolation:** Mocks are required for database interactions during unit testing to isolate business logic.
- **Performance:** Load testing is explicitly excluded from the current phase.

1.4 Definitions, abbreviations and acronyms

- **API:** Application Programming Interface
- **CI:** Continuous Integration (specifically GitHub Actions)
- **ORM:** Object-Relational Mapping (Prisma)
- **JWT:** JSON Web Token (implied by authentication tokens)

2 Quality Assurance Process

2.1 Manual Quality Assurance Process

Manual testing is conducted using Postman to confirm real-world API behavior. This process ensures that:

- Endpoints handle both valid and invalid scenarios correctly.
- Error handling and data validation are accurate.
- Access control is verified using both valid and expired tokens for authenticated routes.

2.2 Automated Quality Assurance Process

Automated testing is integrated into the development workflow via GitHub Actions.

- **Trigger:** Tests run automatically on each push or pull request.
- **Scope:** This includes running all unit tests and validation checks to maintain code quality before merging.

3 Testing process

3.1 General approach

The testing strategy employs a hybrid approach:

1. **Unit Testing:** Focuses on independent controllers and services using Vitest.
2. **Manual Verification:** Uses Postman collections to simulate client usage.
3. **Documentation:** Swagger (OpenAPI 3.0) is used to document and reference all available endpoints.

3.2 Definition of done and goals

The testing process is considered complete when:

- All API endpoints function as intended.

- Automated tests run seamlessly through GitHub Actions.
- Swagger documentation accurately describes all endpoints, including parameters and sample responses.
- Database operations are proven reliable and consistent.

3.3 Testing roles

- **Developers:** Responsible for writing unit tests (Vitest) for their specific controllers and services.
- **Testers/QA:** Responsible for maintaining Postman collections and performing manual validation of routes.

3.4 Test schedule

- **Continuous:** Unit tests run on every code push via GitHub Actions.
- **Manual Cycles:** Performed periodically using Postman to verify end-to-end flows.

3.5 Test documentation

- **Swagger/OpenAPI:** Serves as the primary reference for developers and testers, detailing methods, parameters, and request bodies.
- **Postman Collections:** Act as a repository of executable manual test cases.

4 Test cases

4.1 Unit testing

Unit tests focus on isolating business logic.

- **Scope:** Each controller and service is tested independently.
- **Technique:** Database interactions are mocked to ensure logic is tested without side effects.
- **Coverage:** Includes Authentication, Profiles, Segments, Reviews, and Help Requests services.

4.2 Integration testing

- **API Integration:** Validated via manual Postman tests to ensure the API interacts correctly with the database and authentication middleware.

4.3 System testing

System testing checks the application as a whole through API endpoints.

- **Authentication:** Verifying access control with valid/expired tokens.
- **Error Handling:** Sending invalid data to ensure the system rejects it gracefully.

4.5 Acceptance testing

Based on the provided test case summary, the following scenarios define acceptance:

Module	Test Case	Expected Result
Authentication	Register user	User created successfully
Authentication	Login user	Returns valid token
Authentication	Password reset	Password updated successfully
Profile	Update profile	Returns updated data
Segments	Fetch segments	List retrieved successfully
Reviews	Create review	Review saved correctly
Help Requests	Create request	Request created successfully
Help Responses	Update response	Response status updated

4.6 API Documentation Verification

- Verify that Swagger documentation accurately reflects method types and sample responses for all routes.

5 Adopted Tools

The following tools are adopted for the project's testing and development lifecycle:

1. **Vitest:** For unit testing.
2. **Postman:** For manual API verification and collection management.
3. **GitHub Actions:** For continuous integration and automated test execution.
4. **Swagger:** For API documentation (OpenAPI 3.0).
5. **Prisma ORM:** Managed within tests to ensure database consistency.